T3000 API & Protocol

Message: SAVE_GRAPHIC_DATA · built Nov 13, 2026

Message: SAVE_GRAPHIC_DATA

The SAVE_GRAPHIC_DATA message saves modified graphic screen configurations back to the device. Use this after editing HMI screens in the graphic editor.

When to Use:

- After creating or modifying a graphic screen
- When user clicks "Save" in HMI editor
- To persist graphic element changes

Overview

Action: `SAVE_GRAPHIC_DATA` **Direction:** Frontend → Backend **Location:** BacnetWebView.cpp line 1479

Purpose: Write graphic screen configuration to device memory

Request Format

JSON Structure:

JSON

```
{
  "action": "SAVE_GRAPHIC_DATA",
  "panel_id": 0,
  "graphic_screen_num": 5,
  "data": {
    "screen_name": "Main HVAC",
    "background_color": "#FFFFFF",
    "elements": [
      {
        "type": "TEXT",
        "x": 100,
        "y": 50,
        "width": 200,
        "height": 30,
        "text": "Zone Temperature",
        "font_size": 14,
        "color": "#000000"
      },
      {
        "type": "LIVE_VALUE",
        "x": 320,
        "y": 50,
        "width": 80,
        "height": 30,
        "point_type": "IN",
        "point_index": 5,
        "format": "%.1f",
        "color": "#0000FF"
      }
    ]
  }
}
```

Request Parameters

Parameter	Type	Required	Description
`panel_id`	number	Yes	Target panel ID (0-254)
`graphic_screen_num`	number	Yes	Screen index (1-20)
`data`	object	Yes	Complete screen configuration

Data Object Structure

Field	Type	Description
`screen_name`	string	Screen title (max 31 chars)
`background_color`	string	Hex color (#RRGGBB)
`elements`	array	Array of graphic elements (max 64)

Element Object Structure

Common Fields (all element types):

Field	Type	Required	Description
`type`	string	Yes	Element type (TEXT, LIVE_VALUE, BUTTON, etc.)
`x`	number	Yes	X coordinate (pixels)
`y`	number	Yes	Y coordinate (pixels)
`width`	number	Yes	Width (pixels)
`height`	number	Yes	Height (pixels)

Type-Specific Fields:

TEXT:

- `text` (string): Text content (max 63 chars)
- `font_size` (number): Font size in points
- `color` (string): Text color (#RRGGBB)

LIVE_VALUE:

- `point_type` (string): "IN", "OUT", "VAR", "PRG", "PID", "TL"
- `point_index` (number): Point number (0-63)

- `format` (string): Printf format (e.g., "%.1f", "%d")
- `color` (string): Value color (#RRGGBB)

BUTTON:

- `button_text` (string): Button label (max 31 chars)
- `action_type` (string): "WRITE_VALUE", "TOGGLE", "NAVIGATE"
- `target_point` (number): Point to control (if applicable)

Response Format

Success Response:

```JSON
{
  "action": "SAVE_GRAPHIC_DATA",
  "status": "success",
  "panel_id": 0,
  "graphic_screen_num": 5
}
```

Error Response:

```JSON
{
  "action": "SAVE_GRAPHIC_DATA",
  "status": "error",
  "error": "Too many elements (max 64)",
  "panel_id": 0,
  "graphic_screen_num": 5
}
```

Implementation

Location: BacnetWebView.cpp line 1479

C++

```cpp
case SAVE_GRAPHIC_DATA:
{
    int panel_id;
    int graphic_screen_num;

    // Parse request
    nlohmann::json json_req = nlohmann::json::parse(message);
    panel_id = json_req["panel_id"];
    graphic_screen_num = json_req["graphic_screen_num"];

    // Validate screen number
    if (graphic_screen_num < 1 || graphic_screen_num > MAX_GRPHICS_20)
    {
        // Send error response
        nlohmann::json error_json;
        error_json["action"] = "SAVE_GRAPHIC_DATA";
        error_json["status"] = "error";
        error_json["error"] = "Invalid screen number";
        SendWebViewMessage(error_json.dump().c_str());
        return;
    }

    // Get data object
    nlohmann::json data_json = json_req["data"];

    // Get graphic structure from global array
    Control_group_point& graphic =
        g_GraphicConfigData[panel_id].at(graphic_screen_num - 1);

    // Parse screen properties
    std::string screen_name = data_json["screen_name"];
    strncpy(graphic.screen_name, screen_name.c_str(), 31);
    graphic.screen_name[31] = '\0';

    // Parse background color
    std::string bg_color = data_json["background_color"];
    graphic.background_color = parseHexColor(bg_color);

    // Parse elements
    nlohmann::json elements_array = data_json["elements"];
```

```
int element_count = elements_array.size();

if (element_count > 64)
{
    // Send error response
    nlohmann::json error_json;
    error_json["action"] = "SAVE_GRAPHIC_DATA";
    error_json["status"] = "error";
    error_json["error"] = "Too many elements (max 64)";
    SendWebViewMessage(error_json.dump().c_str());
    return;
}

graphic.element_count = element_count;

for (int i = 0; i < element_count; i++)
{
    nlohmann::json elem_json = elements_array[i];
    Graphic_element& elem = graphic.elements[i];

    // Parse common fields
    std::string type = elem_json["type"];
    elem.element_type = stringToElementType(type);
    elem.x_position = elem_json["x"];
    elem.y_position = elem_json["y"];
    elem.width = elem_json["width"];
    elem.height = elem_json["height"];

    // Parse type-specific fields
    if (elem.element_type == ELEM_TEXT)
    {
        std::string text = elem_json["text"];
        strncpy(elem.text_content, text.c_str(), 63);
        elem.text_content[63] = '\0';

        elem.font_size = elem_json["font_size"];

        std::string color = elem_json["color"];
        elem.text_color = parseHexColor(color);
    }
    else if (elem.element_type == ELEM_LIVE_VALUE)
```

```
    {
        std::string pt_type = elem_json["point_type"];
        elem.point_type = stringToPointType(pt_type);
        elem.point_index = elem_json["point_index"];

        std::string format = elem_json["format"];
        strncpy(elem.value_format, format.c_str(), 7);
        elem.value_format[7] = '\0';

        std::string color = elem_json["color"];
        elem.value_color = parseHexColor(color);
    }
    else if (elem.element_type == ELEM_BUTTON)
    {
        std::string btn_text = elem_json["button_text"];
        strncpy(elem.button_text, btn_text.c_str(), 31);
        elem.button_text[31] = '\0';

        std::string action = elem_json["action_type"];
        elem.action_type = stringToActionType(action);
        elem.target_point = elem_json["target_point"];
    }
}

// Write to device (synchronous BACnet write)
int result = WriteGraphicScreen(
    panel_id,
    graphic_screen_num,
    &graphic,
    sizeof(Control_group_point)
);

// Send response
nlohmann::json response_json;
response_json["action"] = "SAVE_GRAPHIC_DATA";
response_json["panel_id"] = panel_id;
response_json["graphic_screen_num"] = graphic_screen_num;

if (result > 0)
{
    response_json["status"] = "success";
```

```
    }
    else
    {
        response_json["status"] = "error";
        response_json["error"] = "Device write failed";
    }

    SendWebViewMessage(response_json.dump().c_str());
    break;
}
```

Helper Functions

parseHexColor()

Converts hex color string to RGB integer:

C++

```
unsigned long parseHexColor(const std::string& hex_color)
{
    // Remove '#' if present
    std::string hex = hex_color;
    if (hex[0] == '#')
        hex = hex.substr(1);

    // Parse as hex
    unsigned long color = std::stoul(hex, nullptr, 16);
    return color;
}

// Example: "#FF0000" → 0x00FF0000 (red)
```

stringToElementType()

Converts element type string to constant:

C++

```cpp
unsigned char stringToElementType(const std::string& type)
{
    if (type == "TEXT") return ELEM_TEXT;
    if (type == "LIVE_VALUE") return ELEM_LIVE_VALUE;
    if (type == "BUTTON") return ELEM_BUTTON;
    if (type == "IMAGE") return ELEM_IMAGE;
    if (type == "LINE") return ELEM_LINE;
    if (type == "RECTANGLE") return ELEM_RECTANGLE;
    return ELEM_TEXT;  // Default
}
```

Frontend Usage

Save Graphic Screen

TYPESCRIPT

```typescript
const saveGraphicScreen = async (
  panelId: number,
  screenNum: number,
  screenData: GraphicScreenData
) => {
  const message = {
    action: 'SAVE_GRAPHIC_DATA',
    panel_id: panelId,
    graphic_screen_num: screenNum,
    data: {
      screen_name: screenData.screenName,
      background_color: screenData.backgroundColor,
      elements: screenData.elements.map(elem => ({
        type: elem.type,
        x: elem.x,
        y: elem.y,
        width: elem.width,
        height: elem.height,
        ...getTypeSpecificFields(elem)
      }))
    }
  };

  sendMessage(JSON.stringify(message));

  // Wait for response
  return new Promise((resolve, reject) => {
    const handler = (data: string) => {
      const response = JSON.parse(data);

      if (response.action === 'SAVE_GRAPHIC_DATA') {
        if (response.status === 'success') {
          resolve(response);
        } else {
          reject(new Error(response.error));
        }
```

```
        // Remove listener
        webSocket.off('message', handler);
      }
    };

    webSocket.on('message', handler);
  });
};

const getTypeSpecificFields = (elem: GraphicElement) => {
  switch (elem.type) {
    case 'TEXT':
      return {
        text: elem.text,
        font_size: elem.fontSize,
        color: elem.color
      };

    case 'LIVE_VALUE':
      return {
        point_type: elem.pointType,
        point_index: elem.pointIndex,
        format: elem.format,
        color: elem.color
      };

    case 'BUTTON':
      return {
        button_text: elem.buttonText,
        action_type: elem.actionType,
        target_point: elem.targetPoint
      };

    default:
      return {};
  }
};
```

Complete Editor Workflow

TYPESCRIPT

```typescript
// 1. Load existing screen (or create new)
const screenData = await loadGraphicScreen(panelId, screenNum);

// 2. User edits in graphic editor
const editor = new GraphicEditor(screenData);
editor.on('change', () => {
  // Mark as modified
  hasUnsavedChanges = true;
});

// 3. User clicks Save
const handleSave = async () => {
  try {
    const modifiedData = editor.getScreenData();

    await saveGraphicScreen(panelId, screenNum, modifiedData);

    hasUnsavedChanges = false;
    showNotification('Screen saved successfully');
  } catch (error) {
    showError(`Save failed: ${error.message}`);
  }
};
```

Validation

Frontend Validation

Before sending, validate:

TYPESCRIPT

```typescript
const validateGraphicScreen = (data: GraphicScreenData): string[] => {
  const errors: string[] = [];

  // Check screen name length
  if (data.screenName.length > 31) {
    errors.push('Screen name too long (max 31 chars)');
  }

  // Check element count
  if (data.elements.length > 64) {
    errors.push('Too many elements (max 64)');
  }

  // Check each element
  data.elements.forEach((elem, i) => {
    // Check coordinates
    if (elem.x < 0 || elem.y < 0) {
      errors.push(`Element ${i}: negative coordinates`);
    }

    // Check dimensions
    if (elem.width <= 0 || elem.height <= 0) {
      errors.push(`Element ${i}: invalid dimensions`);
    }

    // Type-specific validation
    if (elem.type === 'TEXT' && elem.text.length > 63) {
      errors.push(`Element ${i}: text too long (max 63 chars)`);
    }

    if (elem.type === 'LIVE_VALUE') {
      if (elem.pointIndex < 0 || elem.pointIndex > 63) {
        errors.push(`Element ${i}: invalid point index`);
      }
    }
  });

  return errors;
};
```

```
// Usage
const errors = validateGraphicScreen(screenData);
if (errors.length > 0) {
  showErrors(errors);
  return;
}

// Proceed with save
await saveGraphicScreen(panelId, screenNum, screenData);
```

Performance Characteristics

Data Size:

- Small screen (10 elements): ~2 KB request
- Large screen (64 elements): ~15 KB request

Write Time:

- Average: 200-500ms
- Depends on device response time
- Synchronous operation (blocks until complete)

Recommendations:

- Show loading indicator during save
- Disable editor during save operation
- Implement auto-save with debouncing (save after 2s of inactivity)

Error Handling

Common Errors

Error	Cause	Solution
"Invalid screen number"	Screen num < 1 or > 20	Validate before sending
"Too many elements"	> 64 elements	Limit element count
"Device write failed"	Device offline or timeout	Retry or show error
"Invalid color format"	Bad hex color	Validate color strings

Retry Logic

TYPESCRIPT

```typescript
const saveWithRetry = async (
  panelId: number,
  screenNum: number,
  data: GraphicScreenData,
  maxRetries: number = 3
): Promise<void> => {
  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      await saveGraphicScreen(panelId, screenNum, data);
      return;  // Success
    } catch (error) {
      if (attempt === maxRetries) {
        throw error;  // Final attempt failed
      }

      // Wait before retry (exponential backoff)
      await sleep(1000 * attempt);
    }
  }
};
```

Related Messages

GET_INITIAL_DATA / LOAD_GRAPHIC_ENTRY

Load screen before saving:

TEXT

```
1. GET_INITIAL_DATA → Load existing screen
2. User modifies in editor
3. SAVE_GRAPHIC_DATA → Save changes
```

Workflow Integration

TYPESCRIPT

```typescript
const graphicEditorWorkflow = async (panelId: number, screenNum: number) => {
  // Load
  const data = await loadGraphicScreen(panelId, screenNum);

  // Edit
  const editor = new GraphicEditor(data);
  await editor.waitForUserEdits();

  // Save
  const modifiedData = editor.getScreenData();
  await saveGraphicScreen(panelId, screenNum, modifiedData);

  // Refresh display
  await loadGraphicScreen(panelId, screenNum);
};
```

See Also

- GET_INITIAL_DATA - Loading graphic screens
- Data Structures - Control_group_point structure
- Platform Overview - Architecture